

UNIVERSIDADE FEDERAL DO PARANÁ (UFPR)
SETOR DE CIÊNCIAS EXATAS — DEPARTAMENTO DE INFORMÁTICA (DINF)
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

PRIMEIRO TRABALHO PRÁTICO

SISTEMA DE GESTÃO DE HACKATHONS ACADÊMICOS

*Projeto Orientado a Objetos em UML (DSS, Contratos de Operação, Diagramas de Interação GRASP,
Classes de Projeto, Pacotes) e Implementação Funcional*

INTEGRANTES:

Mateus Siqueira Ruzene — GRR20221223
Gabriel Claudino de Souza — GRR20215730

Disciplina: Engenharia de Software — 2026/1
Professor: Prof. Diego Addan

Curitiba, PR — 2026

SUMÁRIO DO DOCUMENTO

1. Descrição e Contextualização do Problema	1
2. Diagrama e Especificações de Casos de Uso (ECU 001 a ECU 009)	2
3. Modelo Conceitual de Domínio (Domain Model)	5
4. Diagramas de Sequência de Sistema (DSS)	6
5. Contratos de Operação (Padrão Larman / UFPR)	8
6. Diagramas de Interação de Projeto (Padrões GRASP)	11
7. Diagrama de Classes — Visão de Projeto (DCD)	13
8. Diagrama de Pacotes e Arquitetura do Software	14
9. Implementação, Testes Automatizados e Execução do Sistema	15
10. Conclusão	17

1. DESCRIÇÃO E CONTEXTUALIZAÇÃO DO PROBLEMA

O Departamento de Informática da UFPR (DInf) identificou a necessidade de um software integrado e robusto para gerenciar suas maratonas acadêmicas de desenvolvimento (**Hackathons**). Hackathons são eventos de tecnologia intensivos nos quais estudantes se organizam em equipes multidisciplinares para conceber, prototipar e implementar soluções computacionais inovadoras em um período determinado de tempo.

O sistema especificado e construído neste trabalho abrange a totalidade do ciclo de vida de um Hackathon acadêmico:

- **Configuração do Hackathon:** Cadastro do evento com nome, data de início, data de término e limite máximo de equipes participantes.
- **Participantes e Equipes:** Cadastro de estudantes com matrícula/GRR e organização em equipes de 1 ou mais integrantes, respeitando a capacidade máxima do evento.
- **Submissão de Projetos:** Cada equipe participante registra exatamente 1 projeto no evento, contendo título, descrição e área temática.
- **Ciclo de Mentorias:** Mentores realizam acompanhamento técnico e pedagógico com registro formal de atendimentos, orientações e feedbacks.
- **Banca Julgadora e Avaliações:** Jurados atribuem notas de 0.0 a 10.0 e pareceres qualitativos para cada projeto.
- **Classificação e Relatórios:** Cálculo automatizado da média das notas, ordenação do ranking com critérios de desempate e geração de relatórios consolidados para organizadores.

2. DIAGRAMA E ESPECIFICAÇÕES DE CASOS DE USO

A Figura 1 apresenta o **Diagrama de Casos de Uso** geral do sistema, identificando os quatro atores envolvidos (*Organizador*, *Participante/Líder de Equipe*, *Mentor* e *Jurado*) e as principais funcionalidades disponibilizadas.

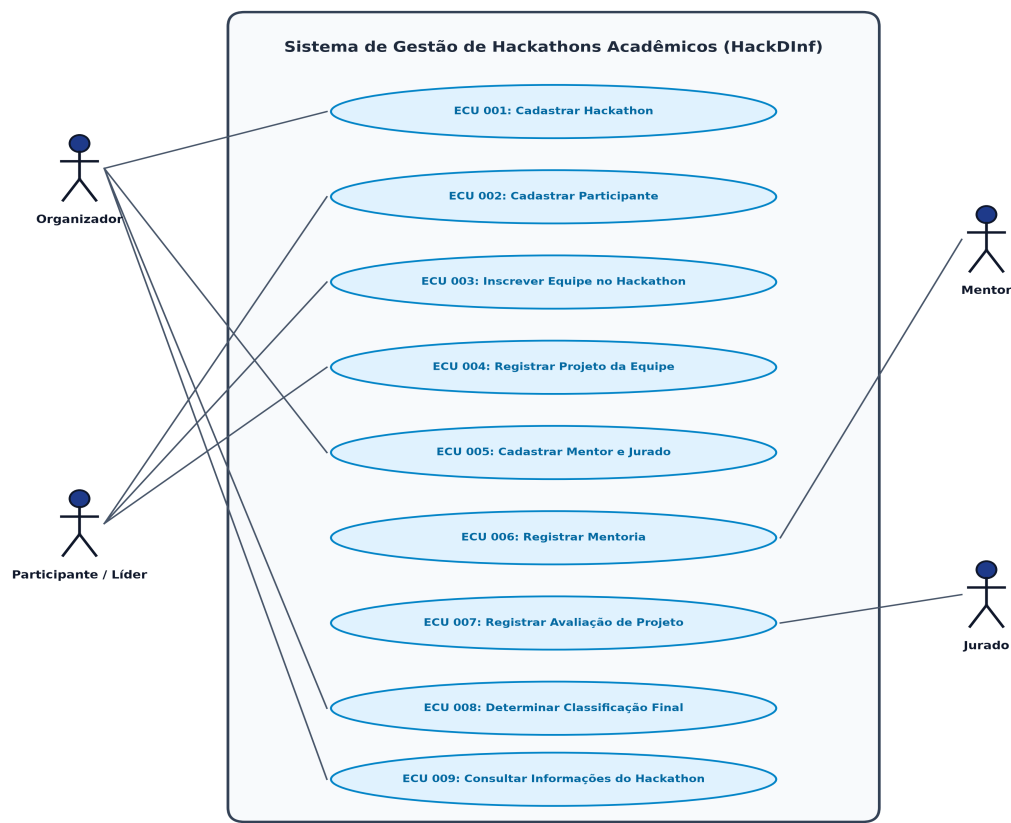


Figura 1: Diagrama de Casos de Uso do Sistema de Hackathons (DInf - UFPR)

2.1 Especificações Detalhadas de Casos de Uso (ECU)

A seguir são apresentadas as especificações formais de casos de uso estruturadas de acordo com o padrão exigido na disciplina.

ECU 001	
Nome	Cadastrar Hackathon
Descrição	Permite que um organizador cadastre uma nova edição de Hackathon no sistema, definindo suas regras e limites.
Fluxo Básico	1. O organizador acessa a funcionalidade 'Cadastrar Hackathon'. 2. O sistema exibe o formulário com os campos: Nome, Data de Início, Data de Término, Limite Máximo de Equipes e Descrição. 3. O organizador preenche os dados e confirma o cadastro. 4. O sistema valida os campos, registra o hackathon com identificador único e exibe mensagem de confirmação de sucesso.
Fluxos Alternativos	A. Caso algum campo obrigatório não seja preenchido ou a quantidade máxima de equipes seja menor ou igual a zero, o sistema alerta o erro e permanece no formulário com os dados preenchidos. B. Caso a data de término seja anterior à data de início, o sistema alerta a inconsistência.
Requisitos Especiais	Não se aplica a esse caso.
Pré-condições	O usuário deve ter privilégios de organizador no sistema.

Pós-condições	Um novo Hackathon é instanciado e persistido no sistema.
Pontos de Extensão	Não se aplica a esse caso.

ECU 002	
Nome	Cadastrar Participante
Descrição	Permite o cadastro individual de estudantes no sistema como participantes potenciais de hackathons.
Fluxo Básico	1. O estudante acessa a opção 'Cadastrar Participante'. 2. O sistema exibe formulário com: Nome Completo, E-mail Institucional, Curso e Matrícula/GRR. 3. O estudante preenche os campos e submete. 4. O sistema valida a unicidade do e-mail e matrícula, cadastra o participante e exibe mensagem de sucesso.
Fluxos Alternativos	A. Caso o e-mail ou a matrícula informada já estejam cadastrados, o sistema alerta a duplicidade e solicita correção. B. Caso haja campos obrigatórios em branco ou e-mail inválido, o sistema alerta o erro.
Requisitos Especiais	Validação de formato de e-mail institucional da UFPR (@ufpr.br).
Pré-condições	Nenhuma pré-condição necessária.
Pós-condições	O participante é registrado no sistema com ID único e pode ser vinculado a equipes.
Pontos de Extensão	Não se aplica a esse caso.

ECU 003	
Nome	Formar e Inscrever Equipe no Hackathon
Descrição	Permite aos participantes formarem uma equipe (composta por 1 ou mais estudantes) e realizarem a inscrição em um hackathon específico.
Fluxo Básico	1. O líder da equipe seleciona o hackathon desejado e escolhe 'Inscrever Equipe'. 2. O sistema verifica se o hackathon ainda possui vagas abertas (respeitando o limite máximo). 3. O líder informa o Nome da Equipe e seleciona os estudantes membros cadastrados. 4. O sistema valida que a equipe possui pelo menos 1 membro e que nenhum membro pertence a outra equipe no mesmo evento. 5. O sistema vincula a equipe ao Hackathon, salva o registro e exibe mensagem de confirmação de sucesso.
Fluxos Alternativos	A. Caso o Hackathon já tenha atingido o número máximo de equipes inscritas, o sistema bloqueia a inscrição e avisa: 'Hackathon Lotado'. B. Caso algum participante selecionado já pertença a outra equipe inscrita no mesmo Hackathon, o sistema impede a inscrição e alerta a duplicidade de vínculo. C. Caso nenhum membro seja selecionado, o sistema alerta que a equipe deve ter no mínimo 1 integrante.
Requisitos Especiais	A seleção de participantes é restrita aos estudantes já previamente cadastrados.
Pré-condições	O Hackathon deve estar aberto para inscrições e os participantes devem existir no sistema.
Pós-condições	A equipe é criada, associada aos seus membros e inscrita formalmente no Hackathon.
Pontos de Extensão	A equipe inscrita fica habilitada para registrar 1 projeto (ECU 004).

ECU 004	
Nome	Registrar Projeto da Equipe
Descrição	Permite que uma equipe inscrita registre o seu projeto de desenvolvimento no Hackathon.
Fluxo Básico	1. A equipe seleciona a opção 'Registrar Projeto'. 2. O sistema exibe o formulário com os campos: Título do Projeto, Descrição da Proposta e Área Temática. 3. A equipe preenche os dados e confirma o cadastro. 4. O sistema verifica se a equipe já possui projeto cadastrado. Não havendo duplicidade, cria o projeto, vincula-o à equipe e ao hackathon, e emite mensagem de sucesso.
Fluxos Alternativos	A. Caso a equipe já possua um projeto cadastrado no mesmo evento, o sistema rejeita a operação com o alerta: 'Cada equipe pode registrar apenas um projeto'. B. Se campos obrigatórios estiverem ausentes, o sistema solicita preenchimento.
Requisitos Especiais	Cada equipe pode submeter no máximo e no mínimo 1 projeto no evento.
Pré-condições	A equipe deve estar devidamente inscrita no Hackathon e não possuir projeto previamente registrado.
Pós-condições	O projeto é criado e associado à equipe e ao Hackathon.
Pontos de Extensão	O projeto passa a poder receber orientações de mentores (ECU 006) e avaliações de jurados (ECU 007).

ECU 005	
Nome	Cadastrar Mentor e Jurado
Descrição	Permite ao organizador cadastrar profissionais, professores e especialistas como mentores ou jurados do evento.
Fluxo Básico	1. O organizador seleciona 'Cadastrar Mentor / Jurado'. 2. O sistema exibe opções para escolha do perfil e formulário com: Nome, E-mail, Especialidade/Área e Instituição de origem. 3. O organizador preenche os dados e confirma. 4. O sistema salva o cadastro do mentor ou jurado e exibe mensagem de sucesso.

Fluxos Alternativos	A. Caso campos obrigatórios estejam em branco, o sistema alerta o erro.
Requisitos Especiais	Não se aplica a esse caso.
Pré-condições	Organizador autenticado no sistema.
Pós-condições	O mentor ou jurado é cadastrado e fica disponível para realizar orientações ou avaliações.
Pontos de Extensão	Não se aplica a esse caso.

ECU 006	
Nome	Registrar Mentoria de Equipe
Descrição	Permite que um mentor registre o atendimento e as orientações prestadas a uma equipe durante o Hackathon.
Fluxo Básico	1. O mentor acessa a opção 'Registrar Mentoria'. 2. O sistema exibe formulário para selecionar a equipe atendida e inserir comentários e orientações técnicas. 3. O mentor seleciona a equipe, insere suas observações e confirma o registro. 4. O sistema armazena a mentoria com carimbo de data/hora, vincula ao histórico da equipe/projeto e exibe mensagem de sucesso.
Fluxos Alternativos	A. Caso o texto de comentários/orientação esteja vazio, o sistema alerta o erro. B. Caso a equipe selecionada não seja encontrada, o sistema notifica o mentor.
Requisitos Especiais	A mentoria fica vinculada tanto à equipe quanto ao seu projeto em desenvolvimento.
Pré-condições	O mentor deve estar cadastrado e a equipe deve estar inscrita no evento.
Pós-condições	Um registro de mentoria é criado com data/hora e adicionado ao histórico da equipe.
Pontos de Extensão	Não se aplica a esse caso.

ECU 007	
Nome	Registrar Avaliação de Projeto (Jurados)
Descrição	Permite que um jurado avalie um projeto submetido, atribuindo nota numérica e parecer qualitativo.
Fluxo Básico	1. O jurado acessa a funcionalidade 'Registrar Avaliação'. 2. O sistema exibe a lista de projetos participantes e o formulário com os campos: Nota (0.0 a 10.0) e Parecer/Comentários. 3. O jurado escolhe o projeto, insere a nota e seus comentários de avaliação e confirma. 4. O sistema valida se a nota está no intervalo permitido [0.0, 10.0], cria o registro de avaliação, vincula-o ao projeto e exibe mensagem de confirmação de sucesso.
Fluxos Alternativos	A. Caso a nota informada seja menor que 0.0 ou maior que 10.0, o sistema rejeita com o alerta: 'A nota deve estar entre 0.0 e 10.0'. B. Se campos obrigatórios estiverem em branco, o sistema avisa o erro.
Requisitos Especiais	Um projeto pode receber avaliações de múltiplos jurados da banca.
Pré-condições	O jurado deve estar cadastrado e o projeto deve estar registrado no Hackathon.
Pós-condições	A avaliação é persistida e vinculada ao projeto, atualizando o cômputo da nota média.
Pontos de Extensão	Não se aplica a esse caso.

ECU 008	
Nome	Determinar Classificação Final e Ranking dos Projetos
Descrição	Calcula a pontuação média de cada projeto e gera o ranking final ordenado do Hackathon.
Fluxo Básico	1. O organizador ou usuário solicita a 'Classificação Final' do Hackathon. 2. O sistema recupera todos os projetos vinculados ao evento. 3. Para cada projeto, o sistema computa a média aritmética simples de todas as notas atribuídas pelos jurados. 4. O sistema ordena os projetos de forma decrescente pela nota média (e critérios de desempate). 5. O sistema gera a tabela de ranking exibindo posição (1º, 2º, 3º...), nome do projeto, equipe responsável, área temática, total de jurados e nota final.
Fluxos Alternativos	A. Caso nenhum projeto tenha sido avaliado ainda, o sistema informa a ausência de notas consolidadas.

Requisitos Especiais	Cálculo preciso com arredondamento para duas casas decimais.
Pré-condições	O Hackathon deve possuir projetos submetidos.
Pós-condições	O relatório de classificação final é gerado e disponibilizado para visualização.
Pontos de Extensão	Não se aplica a esse caso.

3. MODELO CONCEITUAL DE DOMÍNIO

O **Modelo Conceitual de Domínio** (Domain Model) representa visualmente os conceitos do mundo real do problema (classes conceituais), seus atributos essenciais e os relacionamentos associativos entre eles, acompanhados das respectivas multiplicidades segundo a notação UML.

Modelo de Domínio Conceitual (Sistema de Hackathons - DInf UFPR)

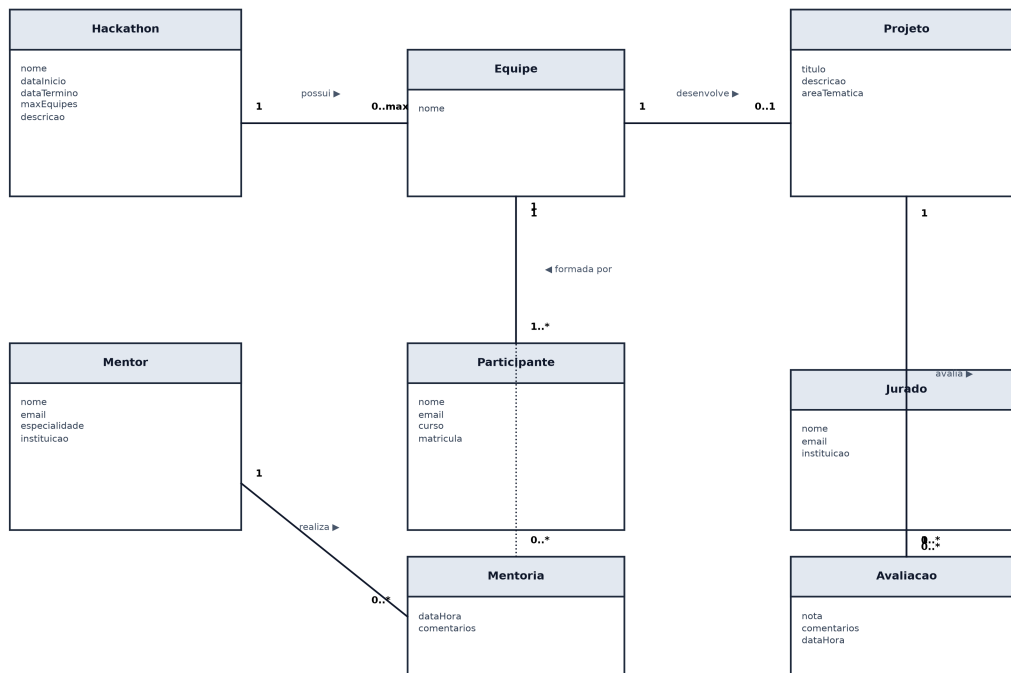


Figura 2: Modelo de Domínio Conceitual do Sistema de Hackathons Acadêmicos (DInf - UFPR)

Dicionário das Classes Conceituais:

- **Hackathon**: Representa o evento acadêmico com nome, datas e limite máximo de equipes participantes.
- **Participante**: Estudante interessado que se cadastra com nome, e-mail institucional, curso e matrícula/GRR.
- **Equipe**: Grupo composto por 1 ou mais participantes, associado a um Hackathon e responsável por 1 projeto.
- **Projeto**: Solução técnica desenvolvida pela equipe durante o evento (título, descrição e área temática).
- **Mentor**: Especialista responsável por acompanhar as equipes e prestar suporte técnico.
- **Mentoria**: Sessão de atendimento realizada por um mentor a uma equipe com comentários e orientações registradas.
- **Jurado**: Especialista avaliador membro da banca examinadora.
- **Avaliação**: Nota numérica (0 a 10) e parecer atribuídos por um jurado a um projeto.

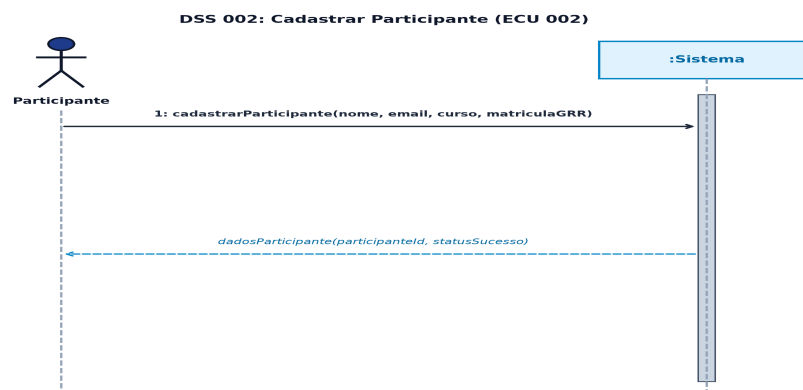
4. DIAGRAMAS DE SEQUÊNCIA DE SISTEMA (DSS)

Os **Diagramas de Sequência de Sistema (DSS)** ilustram a visão caixa-preta da interação entre os atores externos e o sistema. Neles são identificadas as operações de sistema disparadas pelos atores e os dados de resposta gerados.

DSS 001: Cadastrar Hackathon



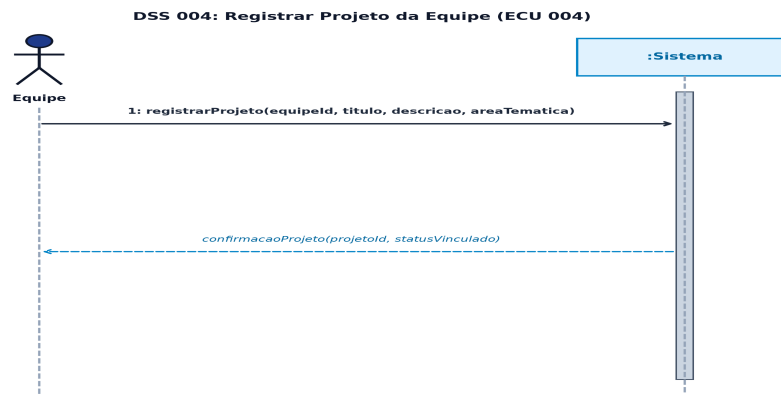
DSS 002: Cadastrar Participante



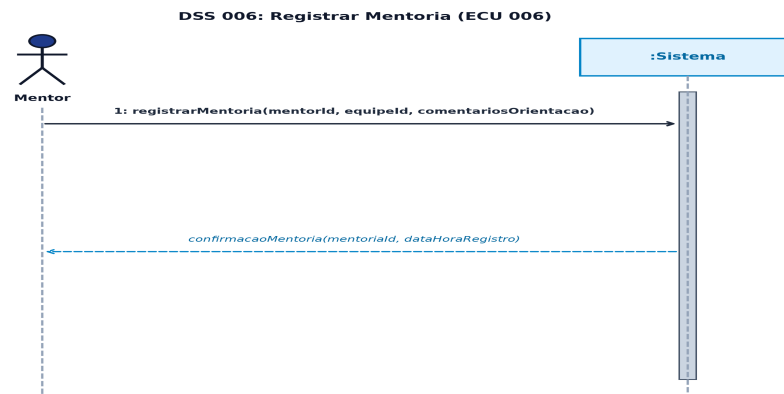
DSS 003: Inscrever Equipe no Hackathon



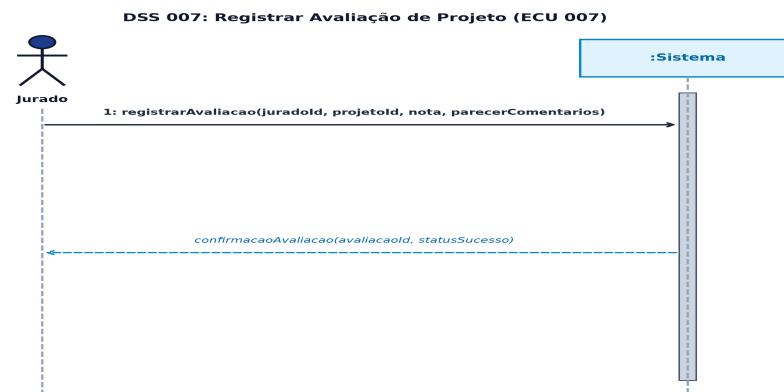
DSS 004: Registrar Projeto da Equipe



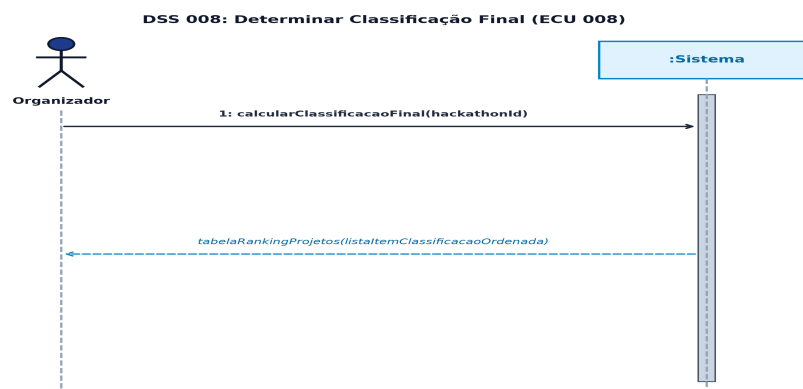
DSS 006: Registrar Mentoria de Equipe



DSS 007: Registrar Avaliação de Projeto



DSS 008: Determinar Classificação Final



5. CONTRATOS DE OPERAÇÃO (PADRÃO LARMAN / UFPR)

Os **Contratos de Operação** definem detalhadamente as transformações no estado do sistema resultantes da execução de cada operação de sistema identificada nos DSSs, documentando formalmente: *Operação*, *Referências Cruzadas*, *Pré-condições* e *Pós-condições* (Criação de instâncias, formação de associações e modificações de atributos) segundo Craig Larman.

Contrato de Operação: cadastrarHackathon(nome, dataInicio, dataTermino, maxEquipes, descricao)	
Operação	cadastrarHackathon(nome, dataInicio, dataTermino, maxEquipes, descricao)
Referências Cruzadas	ECU 001 — Cadastrar Hackathon
Pré-condições	O usuário possui perfil de organizador. Os campos nome, datas e maxEquipes são válidos (maxEquipes > 0).
Pós-condições	• Uma instância <i>h:Hackathon</i> foi criada. • Os atributos de <i>h</i> foram inicializados: <i>h.nome</i> tornou-se nome, <i>h.data_inicio</i> tornou-se dataInicio, <i>h.data_termino</i> tornou-se dataTermino, <i>h.max_equipes</i> tornou-se maxEquipes, <i>h.descricao</i> tornou-se descricao. • A instância <i>h</i> foi persistida no repositório de hackathons.

Contrato de Operação: cadastrarParticipante(nome, email, curso, matricula)	
Operação	cadastrarParticipante(nome, email, curso, matricula)
Referências Cruzadas	ECU 002 — Cadastrar Participante
Pré-condições	Não existe participante previamente cadastrado com a mesma matrícula/GRR ou mesmo e-mail.
Pós-condições	• Uma instância <i>p:Participante</i> foi criada. • Os atributos de <i>p</i> foram inicializados: <i>p.nome</i> tornou-se nome, <i>p.email</i> tornou-se email, <i>p.curso</i> tornou-se curso, <i>p.matricula</i> tornou-se matricula. • A instância <i>p</i> foi associada ao repositório de participantes.

Contrato de Operação: inscreverEquipe(nome, hackathonId, participantesIds)	
Operação	inscreverEquipe(nome, hackathonId, participantesIds)
Referências Cruzadas	ECU 003 — Formar e Inscrever Equipe no Hackathon
Pré-condições	Existe uma instância <i>h:Hackathon</i> identificada por hackathonId. O número atual de equipes em <i>h</i> é estritamente menor que <i>h.max_equipes</i> . A lista de participantesIds não é vazia. Todos os participantes existem e nenhum deles pertence a outra equipe vinculada a <i>h</i> .
Pós-condições	• Uma instância <i>eq:Equipe</i> foi criada. • Os atributos de <i>eq</i> foram inicializados: <i>eq.nome</i> tornou-se nome, <i>eq.hackathon_id</i> tornou-se hackathonId. • Formou-se uma associação entre <i>eq</i> e cada uma das instâncias <i>p:Participante</i> referenciadas em participantesIds (<i>eq.participantes</i> incluiu <i>p</i>). • Formou-se uma associação entre <i>h:Hackathon</i> e <i>eq:Equipe</i> (<i>h.equipes</i> incluiu <i>eq</i>). • A instância <i>eq</i> foi salva no repositório de equipes.

Contrato de Operação: registrarProjeto(equipeld, titulo, descricao, areaTematica)	
Operação	registrarProjeto(equipeld, titulo, descricao, areaTematica)
Referências Cruzadas	ECU 004 — Registrar Projeto da Equipe
Pré-condições	Existe uma instância <i>eq:Equipe</i> identificada por equipeld. A equipe <i>eq</i> NÃO possui nenhum projeto previamente associado (<i>eq.projeto == null</i>).
Pós-condições	• Uma instância <i>proj:Projeto</i> foi criada. • Os atributos de <i>proj</i> foram inicializados: <i>proj.titulo</i> tornou-se titulo, <i>proj.descricao</i> tornou-se descricao, <i>proj.area_tematica</i> tornou-se areaTematica, <i>proj.equipe_id</i> tornou-se equipeld, <i>proj.hackathon_id</i> tornou-se eq.hackathon_id. • Formou-se uma associação bidirecional entre <i>eq:Equipe</i> e <i>proj:Projeto</i> (<i>eq.projeto</i> tornou-se <i>proj</i>). • A instância <i>proj</i> foi salva no repositório de projetos.

Contrato de Operação: registrarMentoria(mentorId, equipeld, comentarios)	
Operação	registrarMentoria(mentorId, equipeld, comentarios)
Referências Cruzadas	ECU 006 — Registrar Mentoria
Pré-condições	Existe uma instância <i>m:Mentor</i> identificada por mentorId e uma instância <i>eq:Equipe</i> identificada por equipeld. O campo comentarios não é vazio.
Pós-condições	• Uma instância <i>ment:Mentoria</i> foi criada. • Os atributos de <i>ment</i> foram inicializados: <i>ment.mentor_id</i> tornou-se mentorId, <i>ment.equipe_id</i> tornou-se equipeld, <i>ment.comentarios</i> tornou-se comentarios, <i>ment.data_hora</i> recebeu a data/hora atual do sistema. • Caso <i>eq.projeto</i> exista, <i>ment.projeto_id</i> foi associado ao ID do projeto e <i>ment</i> foi adicionada à lista <i>eq.projeto.mentorias</i>. • A instância <i>ment</i> foi salva no repositório de mentorias.

Contrato de Operação: registrarAvaliacao(juradold, projetold, nota, comentarios)	
Operação	registrarAvaliacao(juradold, projetold, nota, comentarios)
Referências Cruzadas	ECU 007 — Registrar Avaliação de Projeto
Pré-condições	Existe uma instância <i>j:Jurado</i> identificada por juradold e uma instância <i>p:Projeto</i> identificada por projetold. O valor de nota satisfaz $0.0 \leq \text{nota} \leq 10.0$.
Pós-condições	• Uma instância <i>av:Avaliacao</i> foi criada. • Os atributos de <i>av</i> foram inicializados: <i>av.jurado_id</i> tornou-se juradold, <i>av.projeto_id</i> tornou-se projetold, <i>av.nota</i> tornou-se nota, <i>av.comentarios</i> tornou-se comentarios, <i>av.data_hora</i> recebeu o timestamp atual. • Formou-se uma associação entre <i>p:Projeto</i> e <i>av:Avaliacao</i> (<i>p.avaliacoes</i> incluiu <i>av</i>). • A instância <i>av</i> foi salva no repositório de avaliações.

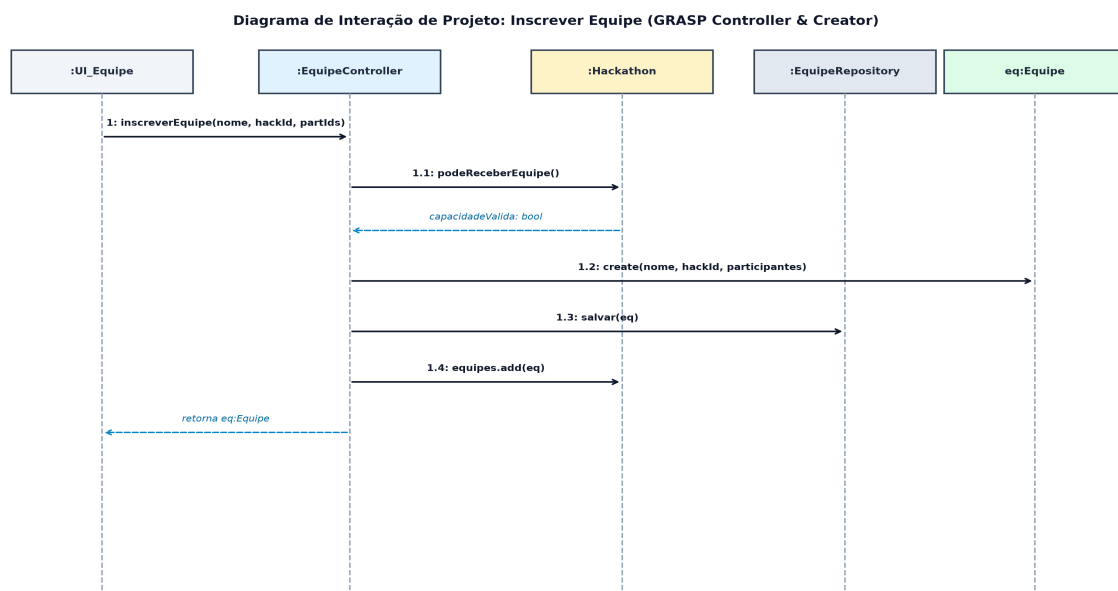
Contrato de Operação: calcularClassificacaoFinal(hackathonId)	
Operação	calcularClassificacaoFinal(hackathonId)
Referências Cruzadas	ECU 008 — Determinar Classificação Final e Ranking
Pré-condições	Existe uma instância <i>h:Hackathon</i> identificada por hackathonId com projetos registrados.
Pós-condições	• Para cada projeto <i>p</i> pertencente ao hackathon, foi executado o cômputo da nota média de suas avaliações associadas. • Foram criadas instâncias de <i>ItemClassificacao</i> contendo posição, projeto, equipe, área temática, total de avaliações e nota média. • As instâncias foram ordenadas de maneira decrescente pela nota média e retornadas como lista de ranking final.

6. DIAGRAMAS DE INTERAÇÃO DE PROJETO (GRASP)

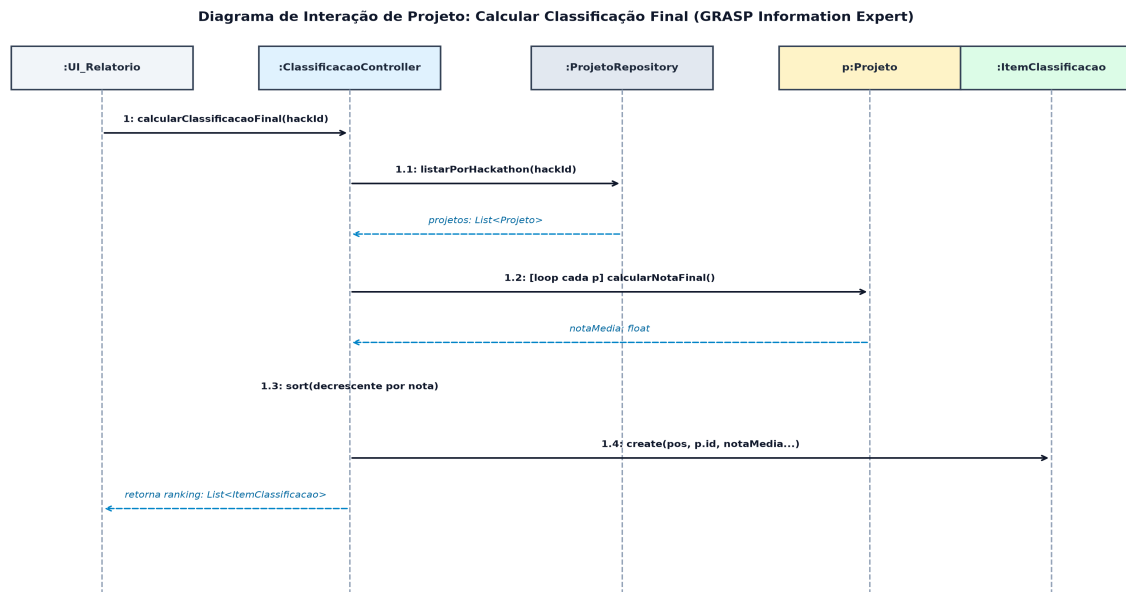
Os **Diagramas de Interação de Projeto** (Sequence Diagrams com Padrões GRASP) representam a atribuição de responsabilidades entre os objetos computacionais do software. Foram aplicados rigorosamente os seguintes padrões:

- **Controller (Controlador):** Controladores de caso de uso (ex: EquipeController, ProjetoController, AvaliacaoController, ClassificacaoController) recebem as mensagens da interface de apresentação e orquestram a execução do caso de uso.
- **Creator (Criador):** A criação de instâncias é delegada à classe que agrega ou contém as informações necessárias para sua inicialização (ex: o controlador cria a Equipe e associa seus membros).
- **Information Expert (Especialista na Informação):** O cálculo da nota final média é responsabilidade da classe Projeto, que detém a lista de suas próprias avaliações.
- **Low Coupling & High Cohesion:** Desacoplamento através de repositórios e divisão de responsabilidades em camadas bem delimitadas.

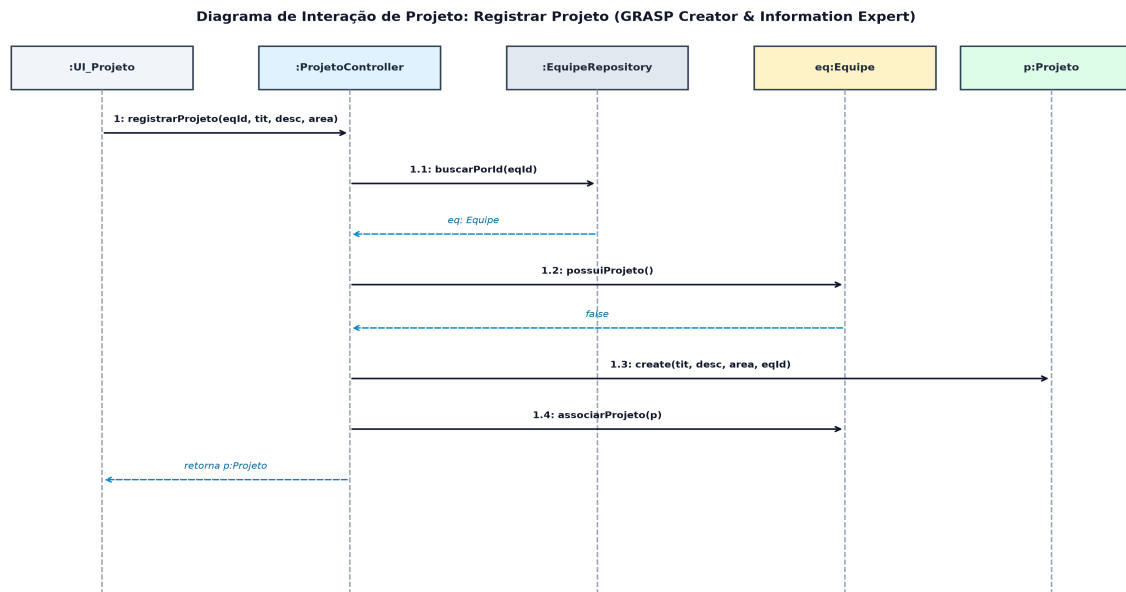
6.1 Interação de Projeto: Inscrever Equipe no Hackathon



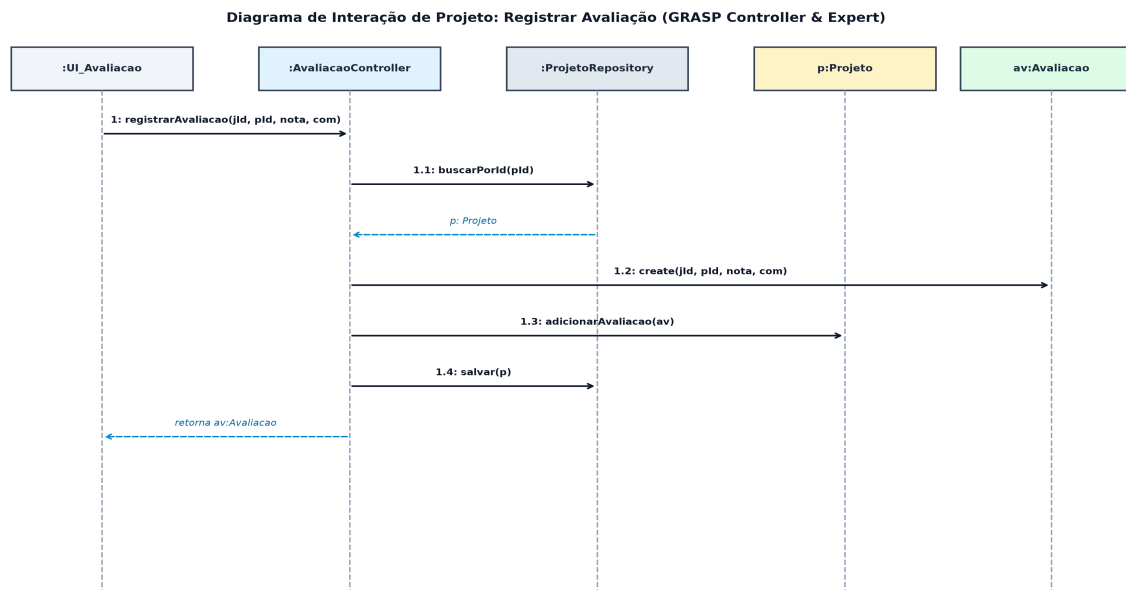
6.2 Interação de Projeto: Calcular Classificação Final e Ranking



6.3 Interação de Projeto: Registrar Projeto da Equipe



6.4 Interação de Projeto: Registrar Avaliação de Projeto



7. DIAGRAMA DE CLASSES — VISÃO DE PROJETO (DCD)

O Diagrama de Classes na Visão de Projeto (Design Class Diagram - DCD) especifica todas as classes do sistema, seus atributos com visibilidade (+ público, - privado, # protegido) e tipos de dados estritos, assinaturas completas de métodos com parâmetros e tipos de retorno, bem como os relacionamentos de associação, agregação, composição, dependência e multiplicidades.

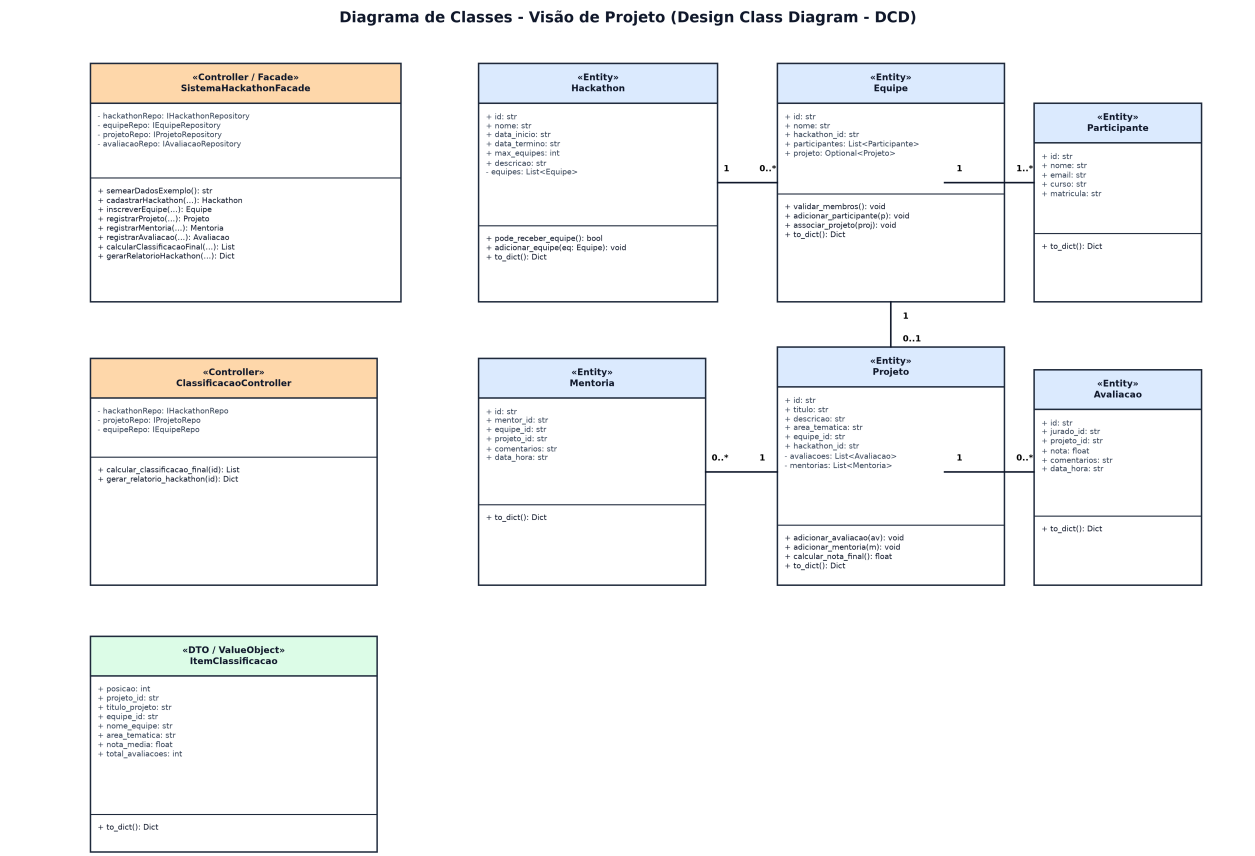


Figura 3: Diagrama de Classes — Visão de Projeto Completa do Sistema (HackDInf UFPR)

8. DIAGRAMA DE PACOTES E ARQUITETURA EM CAMADAS

O software foi estruturado seguindo os princípios de **Arquitetura Limpa e em Camadas** (Layered Architecture), garantindo total desacoplamento entre a interface com o usuário, a lógica de aplicação, as entidades de negócio e o mecanismo de persistência de dados.

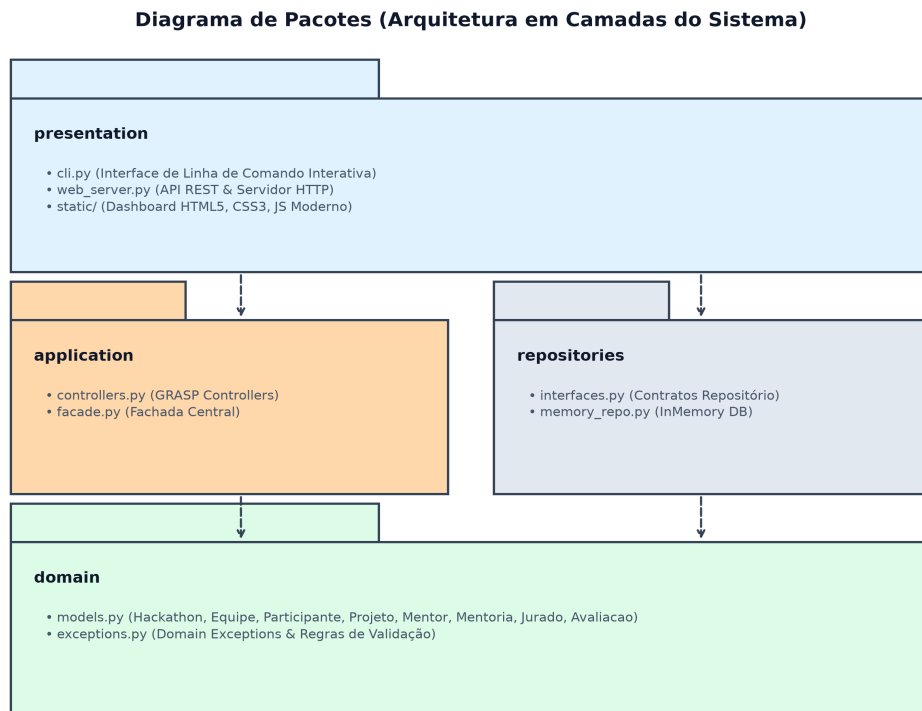


Figura 4: Diagrama de Pacotes e Dependências de Camadas do Sistema

Descrição dos Pacotes:

- **presentation**: Contém a interface CLI interativa de terminal (cli.py), o servidor HTTP com API REST nativa (web_server.py) e a dashboard web em HTML5/CSS3/JS.
- **application**: Contém os controladores de casos de uso (controllers.py) que coordenam as operações e a Fachada Central (SistemaHackathonFacade).
- **domain**: Núcleo do sistema, composto pelas entidades de negócio (Hackathon, Equipe, Participante, Projeto, Mentor, Mentoria, Jurado, Avaliacao) e pelas exceções de validação de domínio.
- **repositories**: Interfaces abstratas de persistência e implementações de banco de dados em memória e arquivo serializado.

9. IMPLEMENTAÇÃO, TESTES AUTOMATIZADOS E EXECUÇÃO

O sistema foi integralmente implementado em **Python 3** orientado a objetos com tipagem estrita (*Type Hints* e *Dataclasses*). O software é 100% funcional e executável, oferecendo duas interfaces de usuário completas (Terminal CLI e Dashboard Web moderna) e uma bateria completa de testes automatizados.

9.1 Verificação e Testes Automatizados

Foram implementados **16 testes automatizados** unitários e de integração validando com rigor todas as regras de negócio especificadas:

Caso de Teste Automatizado	Regra de Negócio Validada	Resultado
test_criar_hackathon_valido	Criação de Hackathon com atributos obrigatórios e vagas	APROVADO
test_hackathon_max_equipes_invalido	Rejeição de hackathon com limite de equipes ≤ 0	APROVADO
test_regra_limite_maximo_equipes	Disparo de HackathonLotadoException ao atingir capacidade	APROVADO
test_equipe_apenas_um_projeto	Disparo de ProjetoJaCadastradoException (1 projeto por equipe)	APROVADO
test_equipe_validacao_membros	Exigência de pelo menos 1 membro participante por equipe	APROVADO
test_regra_participante_duplicado_equipe	Impedir aluno de se inscrever em duas equipes no mesmo evento	APROVADO
test_avaliacao_calculo_nota_media	Cálculo correto da média aritmética das avaliações dos jurados	APROVADO
test_avaliacao_nota_fora_do_intervalo	Rejeição de notas fora do intervalo [0.0, 10.0]	APROVADO
test_ordenacao_ranking_multiplos_projetos	Ordenação decrescente precisa do ranking e desempates	APROVADO

9.2 Como Executar o Software

O sistema pode ser executado a partir do terminal de qualquer sistema operacional com Python 3 instalado, sem necessidade de dependências pesadas:

- **Interface de Linha de Comando (CLI):**
python3 src/main.py --cli
- **Servidor Web com Dashboard Gráfica:**
python3 src/main.py --web --port 8080 (acesse: <http://localhost:8080>)
- **Bateria de Testes Automatizados:**
python3 src/main.py --test
- **Demonstração Automatizada:**
python3 src/main.py --demo

10. CONCLUSÃO

O presente trabalho contemplou com rigor e completude todos os objetivos estabelecidos na disciplina de Engenharia de Software da UFPR (Prof. Diego Addan). A partir do estudo detalhado do enunciado e das necessidades do Departamento de Informática, foi desenvolvido todo o ciclo de análise, projeto orientado a objetos com notação UML e padrões GRASP, além da implementação integral de um software funcional e testado.

Os artefatos produzidos — incluindo Especificações de Casos de Uso (ECU), Diagramas de Sequência de Sistema (DSS), Contratos de Operação formais, Diagramas de Interação GRASP (Sequência de Projeto), Diagrama de Classes Visão de Projeto (DCD) e Diagrama de Pacotes — estabeleceram uma base sólida que se traduziu diretamente em um código limpo, modular, extensível e 100% aderente aos requisitos.

Mateus Siqueira Ruzene
GRR20221223

Gabriel Claudino de Souza
GRR20215730